

Two Ways to Swing Up a Pendulum

Direct collocation and the indirect Pontryagin method, taught side by side

M. Casey (Coorbital, [collocation.examples/](#))

2026-09-17

Abstract

The cart-pole swing-up in `collocation.examples/` is solved twice in this repository: once by *direct collocation* (`ex2_cart_pole_swing_up/`), which chops the trajectory into pieces and hands the whole thing to a nonlinear programming solver, and once by the *indirect* Pontryagin method (`ex3_cart_pole_pmp/`), which writes down the conditions an optimal trajectory must satisfy and then solves those. The two land on the same motion: their costs agree to 0.062% and their control histories to 0.42% RMS.

This document explains both methods, and the bridge between them — the fact that the multipliers the NLP solver produces *are* samples of the costates the Pontryagin method shoots. Every section is given three times: a picture with no mathematics (ELI5), the reasoning an engineer needs in order to trust it (Intuition), and the equations the code implements (Rigor). Code is quoted from the repository as it actually stands, with line-level explanation.

A closing chapter is about a sign error that lived in this code for a long time, what it cost, and the one test that would have caught it on day one.

Contents

1	What problem is this, exactly?	2
2	The plant: a cart, a rod, a bob	3
3	Method 1: direct collocation (discretize, then optimize)	4
4	Method 2: the indirect method (optimize, then discretize)	6
5	The bridge: NLP multipliers <i>are</i> costates	7
6	Multiple shooting and the state-transition matrix	8
7	What the two methods produce	10
8	How we know it is right	10
9	A sign error, and the test that would have caught it	11
10	Where the code lives	12
11	The executable companion, and the objective that never has to choose	13

1 What problem is this, exactly?

Before any method: *which* optimal control problem is being solved? The three classical choices lead to very different mathematics, and calling this one by the wrong name will send you looking for structure that is not there.

ELI5

We want to swing a pendulum from hanging straight down to balanced straight up, in exactly five seconds, by sliding the cart it is attached to back and forth. Lots of different pushes would get it there. We want the *gentlest* one — the one that uses the least force overall. We are not in a hurry (five seconds is fixed, not something we are trying to shrink), and we are not paying for fuel by the litre. We are paying for *effort*.

Intuition

Three costs, three worlds:

- **Minimum time** $J = t_f$: “get there as fast as possible”. The final time is an unknown, and the answer almost always pushes as hard as the limits allow. The interesting structure is *when* to switch between limits.
- **Minimum fuel** $J = \int |u| dt$: “spend as little propellant as possible”. Because $|u|$ is linear in the control’s magnitude, the optimum is bang-bang: full on or full off, with a switching function deciding which. This is the world where you need saltation matrices, smoothing homotopies, and all the switch-detection machinery in `orbit_transfer/`.
- **Minimum energy (effort)** $J = \int u^2 dt$: “push as gently as possible”. The cost is *quadratic*, so the optimal control comes out as a smooth formula rather than a switching rule. Nothing bangs; nothing switches.

Ours is the third, with t_f fixed at 5 s. That choice is what makes this problem a good teaching example: the optimality condition can be solved for u in closed form, so the only hard part left is the boundary-value problem — which is exactly the part we want to teach.

Rigor

The problem solved by both `ex2` and `ex3` is

$$\min_{u(\cdot)} J = \int_0^{t_f} u(t)^2 dt, \quad t_f = 5 \text{ s fixed},$$

subject to $\dot{x} = f(x) + g(x)u$ with $x = [q_1, q_2, \dot{q}_1, \dot{q}_2]^\top$ and boundary conditions

$$x(0) = [0, 0, 0, 0]^\top, \quad x(t_f) = [0, \pi, 0, 0]^\top,$$

i.e. hanging at rest to upright at rest, cart returning to the origin. The control is unconstrained: no bound enters the formulation. (The direct fixture is solved at a deliberately loose bound of 2000 N purely so that the bound is inactive — measured $\max |u| = 68.5$ N — and it therefore approximates the same unconstrained optimum.)

Naming: in the literature this is *minimum energy* or *minimum effort*; both names appear. “Energy” here refers to the *control* energy $\int u^2$, which is not the mechanical energy of the cart-pole. The mechanical energy makes its own appearance in §9, as a test.

Where it breaks / what is assumed

The word “energy” is overloaded. $\int u^2 dt$ has units of $N^2 s$ — it is not joules, and it is not the work done by the force ($\int u \dot{q}_1 dt$, which *is* joules and *is* the quantity the physics test in §9 uses). Keep them apart.

2 The plant: a cart, a rod, a bob

ELI5

A cart slides on a rail. A rigid rod hangs off it with a weight on the end. You can push the cart left or right; you cannot touch the rod. Everything the rod does, it does because the cart moved underneath it — like balancing a broom on your palm, except your palm may only slide along a straight line.

Intuition

Two degrees of freedom (q_1 , the cart’s position; q_2 , the rod’s angle) and only one actuator (the force u on the cart). That deficit — more things to control than knobs to control them with — is what makes this *underactuated*, and it is why the answer has to be a swing: the only way to put energy into the rod is to move the cart so that the rod is dragged, and the only way to stop the rod at the top is to move the cart out from under it at exactly the right moment.

The single most important structural fact for everything that follows is that the force enters *linearly*. Push twice as hard and the acceleration contribution doubles exactly. Formally $\dot{x} = f(x) + g(x)u$: a part that happens anyway (the drift f , gravity and centrifugal terms) plus a part proportional to what you do (g , the control column). Almost every nice property we exploit downstream traces back to that split.

Rigor

With q_2 measured from *hanging*, the bob sits at $(q_1 + L \sin q_2, -L \cos q_2)$, and the Lagrangian equations give

$$f(x) = \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \frac{L m_2 s \dot{q}_2^2 + m_2 g c s}{D_1} \\ -\frac{L m_2 c s \dot{q}_2^2 + (m_1 + m_2) g s}{D_2} \end{bmatrix}, \quad g(x) = \begin{bmatrix} 0 \\ 0 \\ \frac{1}{D_1} \\ -\frac{c}{D_2} \end{bmatrix},$$

with $s = \sin q_2$, $c = \cos q_2$, $D_1 = m_1 + m_2(1 - c^2) = m_1 + m_2 s^2$ and $D_2 = L(m_1 + m_2)(1 - \frac{m_2}{m_1 + m_2} c^2) = L D_1$. Constants: $m_1 = 5$ kg, $m_2 = 1$ kg, $L = 2$ m, $g = 9.8$ m/s².

Note the minus signs on the fourth row of both f and g . They are the subject of §9.

In code, `cartpole_common/cartpole_field.m` is exactly this split:

```

1  q1d = x(3); q2d = x(4);
2  s = sin(x(2)); c = cos(x(2));
3  D1 = p.m1 + p.m2*(1 - c^2);
4  D2 = p.L*(p.m1 + p.m2)*(1 - (p.m2/(p.m1 + p.m2))*c^2);
5
6  F = [q1d;
7       q2d;
8       (p.L*p.m2*s*q2d^2 + p.m2*p.g*c*s)/D1;
9       -(p.L*p.m2*c*s*q2d^2 + (p.m1 + p.m2)*p.g*s)/D2];
10 G = [0; 0; 1/D1; -c/D2];

```

Two details that matter later. First, F is the derivative at $u = 0$ and G is exactly $\partial \dot{x} / \partial u$, so $F + G*u$ is the full dynamics — the test suite checks this against an independently written copy of the equations rather than trusting the claim. Second, the file contains no `abs`, `norm`, `max`, `min`, or branch on a state value. That makes it *complex-step safe*, which §6 cashes in.

3 Method 1: direct collocation (discretize, then optimize)

ELI5

Guess the whole movie at once. Chop the five seconds into 200 slices and write down, for every slice boundary, where the cart is, how fast it is going, where the rod is pointing, and how hard you are pushing. That is a big list of numbers — about a thousand of them. Now hand the list to a solver and say: “adjust these until (a) the pieces fit together like a physically possible movie, (b) the start and end look the way I asked, and (c) the total push is as small as you can make it.”

The solver does not know any physics. It only knows the rules you wrote down.

Intuition

The trick that makes this work is how “physically possible” is expressed. You do *not* integrate the equations of motion. Instead you write, for each slice, an equation that says: *the change in the state across this slice equals the average of the slopes at its two ends, times the slice width*. That is the trapezoid rule, turned from an integration scheme into a constraint.

Each such equation is called a *defect*: it is zero when the piece is consistent, nonzero when it is not. The solver’s job is to drive all of them to zero while minimizing cost. The states and controls are all decision variables on an equal footing, which is why this family is called *simultaneous* or *discretize-then-optimize*.

Why it is popular: it is robust, it needs no good initial guess for the costates (there are none), and bounds on states or controls are just bounds on variables. Why it has a ceiling: the answer is only as good as the mesh, and the optimality it achieves is the optimality of the *discretized* problem, which is not quite the problem you asked about.

Rigor

Let N be the number of segments, $h = t_f/N$, and let x_k, u_k denote the state and control at node $k = 1, \dots, N + 1$. The decision vector stacks them by variable:

$$Z = [q_1^{(1..N+1)}, q_2^{(1..N+1)}, \dot{q}_1^{(1..N+1)}, \dot{q}_2^{(1..N+1)}, u^{(1..N+1)}]^\top \in \mathbb{R}^{5(N+1)}.$$

The trapezoidal defect for segment k is

$$\zeta_k = x_{k+1} - x_k - \frac{h}{2} [\hat{f}(x_{k+1}, u_{k+1}) + \hat{f}(x_k, u_k)] = 0, \quad \hat{f} = f + g u,$$

and the cost is the trapezoid quadrature of the running cost,

$$J \approx \frac{h}{2} \sum_{k=1}^N (u_k^2 + u_{k+1}^2).$$

Together with the eight boundary conditions this is a nonlinear program in $\mathbb{R}^{5(N+1)}$ with $4N + 8$ equality constraints, handed to `fmincon`.

The defects, from `ex2_cart_pole_swing_up/try2/nonlcon_cart_pole.m`, written vectorized over all segments at once:

```
1 % kinematic rows: change in position = average velocity x h
2 ceq1 = q1(2:end) - q1(1:end-1) - h/2.*(q1dot(2:end) + q1dot(1:end-1));
3 ceq2 = q2(2:end) - q2(1:end-1) - h/2.*(q2dot(2:end) + q2dot(1:end-1));
4
5 % dynamic rows: change in velocity = average acceleration x h
6 term_kp1_cart_accel = cart_accel(q2(2:end), q2dot(2:end), u(2:end), L, m1, m2, g);
7 term_k_cart_accel = cart_accel(q2(1:end-1), q2dot(1:end-1), u(1:end-1), L, m1, m2, g);
8 ceq3 = q1dot(2:end) - q1dot(1:end-1) - h/2.*(term_kp1_cart_accel + term_k_cart_accel);
```

Line by line: `q1(2:end)` is every node from the second onward and `q1(1:end-1)` every node up to the second-to-last, so their difference is the vector of changes across all N segments in one shot. Subtracting $h/2$ times the sum of the endpoint slopes is precisely ζ_k for every k . Rows `ceq1/ceq2` use velocity as the slope of position (that part is exact kinematics); `ceq3/ceq4` call the acceleration helpers for the slope of velocity.

And the objective, from `objective.m`:

```
1 J = h/2 .* sum (u(2:end).^2 + u(1:end-1).^2,1);
```

which is the quadrature above, again vectorized over segments.

Where it breaks / what is assumed

A converged NLP means *the defects are small*, not *the trajectory is accurate*. These are different claims, and on stiff problems they can differ by many orders of magnitude — the orbit-transfer library has a documented case of a defect of 1.4×10^{-14} next to a true local error of 1.5. The honest check is to re-integrate the reconstructed control with a real integrator and see where you actually arrive; `oc.local_residual` and `oc.fly_control` exist for exactly this.

4 Method 2: the indirect method (optimize, then discretize)

ELI5

Instead of guessing the whole movie, ask a different question: *what must be true of the best movie?* Physics and calculus answer that for you. It turns out that alongside the four numbers describing the cart and rod, there are four more shadow numbers — one attached to each — that tell you how much the final result would improve if that quantity were nudged. Those shadows obey their own equations, running alongside the real ones.

Here is the magic: once you know the four shadows at the *start*, the best push at every instant is given by a formula, and the whole movie follows by playing the equations forward. So the entire problem shrinks from a thousand unknowns to four. The catch is that you do not know those four numbers — you have to hunt for the ones that make the rod finish upright.

Intuition

The shadow variables are the *costates* λ , and they are Lagrange multipliers for the dynamics — the price of the constraint “the state must actually obey the equations of motion”. Pontryagin’s Minimum Principle says an optimal trajectory makes a certain scalar, the Hamiltonian

$$H = \underbrace{u^2}_{\text{what this instant costs}} + \underbrace{\lambda^\top (f + gu)}_{\text{what this instant buys}},$$

as small as possible *at each instant*, with respect to u alone.

Because our cost is quadratic in u and the dynamics are linear in u , H is a quadratic in u with a positive leading coefficient — a parabola opening upward. Minimizing a parabola is done by setting its derivative to zero, which gives u in closed form. This is the sense in which minimum-energy problems are *easy*: there is no switching, no case analysis, no bang–bang. Change the cost to $\int |u|$ and the parabola becomes a V with a flat bottom, and all the switching machinery appears.

So the method is: eliminate u analytically, leaving an eight-dimensional flow in (x, λ) ; then find the four unknown initial costates that make the terminal conditions come out right. Four unknowns, four equations. That is a root-find, not an optimization — which is why it converges quadratically when it converges, and why it can be delicate.

Rigor

With $H = u^2 + \lambda^\top (f + gu)$, the three PMP conditions are

$$\dot{x} = \frac{\partial H}{\partial \lambda} = f + gu, \quad \dot{\lambda} = -\frac{\partial H}{\partial x}, \quad u^* = \arg \min_u H.$$

Stationarity gives $\partial H / \partial u = 2u + \lambda^\top g = 0$, hence

$$\boxed{u^* = -\frac{1}{2}\lambda^\top g} \quad \text{and} \quad \frac{\partial^2 H}{\partial u^2} = 2 > 0,$$

so the stationary point is the *pointwise minimizer* everywhere (the Legendre–Clebsch condition holds strictly). Substituting u^* closes an autonomous 8-state system in $y = [x; \lambda]$.

The costate equation deserves care. After substituting $u^*(x, \lambda)$ the Hamiltonian becomes $\bar{H}(x, \lambda) = H(x, \lambda, u^*)$, and

$$\bar{H}_x = H_x + H_u u_x^* = H_x \quad \text{because } H_u = 0 \text{ at } u^*.$$

So one may differentiate *holding u fixed* and then evaluate at u^* — the $\partial u^*/\partial x$ term drops out exactly. Since the running cost u^2 has no state dependence,

$$\dot{\lambda} = -A(x, u^*)^\top \lambda, \quad A = \left. \frac{\partial(f + gu)}{\partial x} \right|_{u \text{ fixed}}.$$

Boundary conditions: $x(0)$ is fully specified (4 conditions) and $x(t_f)$ is fully specified (4 more). With t_f fixed there is no transversality condition on t_f , and because the terminal state is fully constrained there is no condition $\lambda(t_f) = 0$ either. The unknowns are exactly $\lambda(0) \in \mathbb{R}^4$.

The whole of `cartpole_pmp_rhs.m` is four lines:

```
1 x = y(1:4); lam = y(5:8);
2 [F, G] = cartpole_field(x, p);
3 u = -(lam.'*G)/2;
4 A = cartpole_state_jac(x, u, p);
5 dy = [F + G*u; -A.'*lam];
```

Line 3 is u^* . Line 4 evaluates A at that control — fixed- u differentiation, per the argument above. Line 5 stacks $\dot{x}$ on top of $\dot{\lambda}$. Note `.'` (transpose) rather than `'` (conjugate transpose) in both places: on a complex-stepped path a conjugate transpose would silently destroy the derivative.

Where it breaks / what is assumed

$H_{uu} > 0$ makes u^* the unique minimizer *at each instant*. It does *not* make the resulting trajectory a minimizing one. The boundary-value problem can have several extremals, saddles among them; ruling those out needs a second-order test (a conjugate-point or Jacobi test — see `costate_common/ms_conjugate_test.m` in the orbit-transfer library). Everything this demo verifies is consistent with a *normal extremal*, which is a weaker statement than “the optimum”.

5 The bridge: NLP multipliers *are* costates

This is the conceptual heart of the pairing, and the reason the two examples live next to each other.

ELI5

The first method never mentions shadow numbers. But it computes them anyway, without being asked — they fall out of how the solver enforces “the pieces must fit”. So you can run the easy, robust method, read the shadows out of its answer, and hand them to the fussy, precise method as a starting guess. Easy method warms up the hard one.

Intuition

When `fmincon` enforces the defect $\zeta_k = 0$, it attaches a Lagrange multiplier μ_k to it. Stationarity of the NLP’s own Lagrangian with respect to the state variables is, term by term, a discretization of the adjoint equation $\dot{\lambda} = -H_x$. So μ_k is a sample of λ — up to a scaling that depends on how the defect was written, a sign that depends on the solver’s conventions, and a *station*: for the trapezoid rule the natural association is with nodes, for Hermite–Simpson with interval midpoints.

Getting that association wrong injects a half-step phase error into every seed, and it is a genuinely easy mistake — it has been made in this codebase and caught by review. That is why the rules live in one place, `oclib/+oc/duals_to_costates.m`, rather than being re-derived per campaign.

Rigor

The seed construction in `run_cartpole_pmp.m`:

```
1 [lamS, tS] = oc.duals_to_costates(struct('scheme', 'trapezoid', 'mu', R.muDefect, ...
2                                     'tNodes', R.tN, 'velRows', 3:4));
```

`'scheme', 'trapezoid'` selects the station rule; `mu` is the $4 \times N$ array of defect multipliers harvested from the direct solve; `velRows` names which rows hold the velocity costates (the default 4:6 is for the 7-state CR3BP problems, and would be wrong here).

The global sign is convention-dependent. The library resolves it by a “primer vote”: the implied thrust direction $-\lambda_v/\|\lambda_v\|$ should align with the thrust direction the solver actually used. A scalar force has no direction to vote on, so this demo does the scalar analogue — and, after review, does it with a confidence measure rather than a bare sign test:

```
1 signCorr = sum(uImplied.*uDirect) / sqrt(sum(uImplied.^2)*sum(uDirect.^2));
2 assert(abs(signCorr) > 0.5, 'run_cartpole_pmp:sign', ...
3       'implied vs direct control correlation %.3f: the seed carries no usable sign
4       information', signCorr);
5 if signCorr < 0, lamS = -lamS; uImplied = -uImplied; end
```

The normalized correlation is a real diagnostic: measured 0.9998 here. A bare $\sum u_{\text{implied}} u_{\text{direct}} \neq 0$ test would have accepted an arbitrarily weak agreement. The code also reports the *amplitude ratio* (1.009 measured), because a sign vote cannot detect a costate seed that points the right way but is scaled wrongly.

6 Multiple shooting and the state-transition matrix

ELI5

Playing the equations forward for five seconds from a guessed start is like rolling a marble down a long bumpy hill and hoping it lands in a cup: tiny errors at the top become huge misses at the bottom. So we cut the five seconds into eight shorter rolls, let each one start wherever it likes, and add the rule “where one roll ends is where the next begins”. More unknowns, but each roll is short enough to be predictable.

To steer the hunt we also need to know how the landing point moves when the starting point moves. That sensitivity is a matrix, and we compute it by carrying it along with the

trajectory.

Intuition

The reason single shooting fails on long arcs is Lyapunov amplification: the sensitivity of the endpoint to the initial costate grows roughly exponentially with the arc length. In the orbit campaigns, catalog seeds that are perfectly good miss by 36,000 to 560,000 km when single-shot. Splitting the horizon into K segments replaces one badly-conditioned map with K well-conditioned ones, at the price of carrying the junction states as extra unknowns and adding continuity constraints.

The Newton step needs the Jacobian of “miss” with respect to the unknowns, and the pieces of that Jacobian are *state-transition matrices*: $\Phi = \partial y(t_{k+1})/\partial y(t_k)$. You get Φ by integrating its own differential equation alongside the state, $\dot{\Phi} = A_8\Phi$, starting from the identity.

Rigor

The engine (`oclib/+oc/ms.bvp.m`) asks the problem for three closures: `prop` (propagate, optionally with Φ), `rhs` (the field at a point), and `terminal` (the terminal residual and its Jacobian). The demo supplies them:

```
1 prob = struct('ny', 8, 'freeIdx0', 5:8, ...
2   'prop', @(dt, y0, needSTM) cartpole_pmp_prop(dt, y0, needSTM, p), ...
3   'rhs', @(y) cartpole_pmp_rhs(y, p), ...
4   'terminal', @(y, needJ) terminalFcn(y, xf, needJ));
```

`'ny', 8` is the augmented state dimension; `'freeIdx0', 5:8` says that of the eight components at $t = 0$, only rows 5–8 (the costates) are unknowns — rows 1–4 are the fixed initial state. With `opts.fixedTf` the unknown vector is $p = [\lambda(0); y_2; \dots; y_K]$ and the equations are the $K - 1$ continuity blocks plus the 4 terminal conditions.

The variational integration, from `cartpole_pmp.prop.m`:

```
1 y = z(1:8);
2 PHI = reshape(z(9:72), 8, 8);
3 dy = cartpole_pmp_rhs(y, p);
4 A = zeros(8);
5 h = 1e-20;
6 for col = 1:8
7     yp = y; yp(col) = yp(col) + 1i*h;
8     A(:,col) = imag(cartpole_pmp_rhs(yp, p))/h;
9 end
10 dz = [dy; reshape(A*PHI, 64, 1)];
```

Seventy-two equations: eight for y , sixty-four for Φ . The Jacobian A comes from a *complex step*: perturb one component by ih with $h = 10^{-20}$ and read the imaginary part of the output. Unlike a finite difference this has no subtractive cancellation, so it is accurate to machine precision even at absurdly small h .

Which raises the subtlety that dictated the design of `cartpole_state_jac.m`. The complex step above runs *through* `cartpole_pmp_rhs`, which itself needs A for the costate equation. Had that inner A also been computed by a complex step, the two would share the same imaginary channel and the inner derivative would be silently corrupted — no error, no

warning, just wrong numbers. That is why the inner Jacobian is generated symbolically instead:

```

1 syms q1 q2 q1d q2d u m1 m2 L g real
2 xs = [q1; q2; q1d; q2d];
3 ps = struct('m1', m1, 'm2', m2, 'L', L, 'g', g);
4 [F, G] = cartpole_field(xs, ps); % the FIELD ITSELF, on symbols
5 A = simplify(jacobian(F + G*u, xs));
6 matlabFunction(A, 'File', out, 'Vars', {xs, u, [m1; m2; L; g]}, ...);

```

The generator calls `cartpole_field` on symbolic inputs, so there is no second copy of the physics to drift out of step — a lesson learned the hard way (§9).

7 What the two methods produce

Both solve the same problem; the comparison is the point.

	Direct (ex2 formulation)	Indirect (ex3)
unknowns	1005	4 (plus 7×8 junction states)
cost J	2781.109846	2779.381719
terminal miss	pinned by constraint	3.67×10^{-11}
needs a costate guess	no	yes
handles bounds	trivially	needs switching structure
accuracy limit	the mesh	the integrator

The costs differ by 0.062%, and the control histories by 0.42% RMS. The gap is the direct method's discretization error: the indirect solve is the more accurate of the two, because it never discretizes the dynamics at all — it integrates them.

Intuition

The practical division of labour, and the reason the orbit-transfer library uses both: the direct method is *robust but coarse*, the indirect method is *sharp but fragile*. So run the direct method to get into the right basin, harvest its multipliers (§5), and let the indirect method polish the answer to integrator accuracy. That is precisely the pipeline the costate catalogs use, at scale, on orbit transfers.

8 How we know it is right

ELI5

Getting a pretty answer is easy. Knowing it is the right answer is the job. We check four different things, and each one can fail on its own.

Intuition

Verification has layers, and they catch different faults:

1. **Did the root-find converge?** The engine's own residual. Necessary, and nearly

meaningless by itself.

2. **Does the trajectory hit the target?** Re-fly $\lambda(0)$ from scratch and measure the terminal miss. This catches a converged solve of the *wrong* equations.
3. **Is it an extremal?** Along an autonomous, fixed- t_f extremal the Hamiltonian is *constant* (not zero — that would require free t_f). Measured spread: 5.8×10^{-13} relative.
4. **Does the control actually fly?** Take $u(t)$ and integrate the true dynamics with it. Measured arrival error: 1.7×10^{-7} .

A trap worth naming: a check that recomputes the thing it is testing proves nothing. The stationarity residual $|2u + \lambda^\top g|$ is exactly zero when u was *built* as $-\lambda^\top g/2$ — it measures arithmetic, not optimality. This demo used to report that number as evidence; it no longer does.

Rigor

The engine's own residual and the reported miss are different quantities and were confused here once. Measured on the converged root: the engine's last-arc terminal residual is 7.27×10^{-14} , while re-flying $\lambda(0)$ over the whole horizon gives 6.88×10^{-9} at **RelTo1** 10^{-12} , 8.20×10^{-10} at 10^{-13} , and 3.67×10^{-11} at 10^{-14} . The loose figure was the *measurement*, not the solution — the reporting integration's own error. The fix was to tighten the measurement, not the gate.

9 A sign error, and the test that would have caught it

ELI5

For a long time the equations in this repository had one minus sign missing. It made the hanging position *unstable* — in that world, a pendulum hanging straight down would fall *upwards*. Both methods solved that world happily, and agreed with each other beautifully, because they were both asking the same wrong question. Agreeing with yourself is not evidence.

Intuition

Every test in the folder was *relative*: the field was checked against a copy of the equations, the Jacobian against the field, the STM against the flow, the indirect solve against the direct one. A fault shared by all of them cancels everywhere and is invisible.

What breaks the circle is a test whose reference comes from *outside* the equations under test. For mechanics the cheapest such reference is a conservation law. You can write the energy from the *geometry* alone — where the masses are and how fast they move — without consulting the acceleration formulas at all. Then physics demands a relationship between that energy and the work being done, and any sign error in the accelerations breaks it.

Rigor

Build the energy from positions only. The bob is at $(q_1 + L \sin q_2, -L \cos q_2)$, so

$$E = \frac{1}{2}m_1\dot{q}_1^2 + \frac{1}{2}m_2\|\dot{r}_{\text{bob}}\|^2 - m_2gL \cos q_2,$$

and the only external force doing work is u acting on the cart, giving the *pointwise* identity

$$\frac{dE}{dt} = \nabla E \cdot \dot{x} = u \dot{q}_1 \quad \text{for every } (x, u).$$

No integration is needed: evaluate both sides at a scatter of states and controls. From `tests/test_cartpole_physics.m`:

```
1 gradE = energy_gradient(x, p); % complex step through the GEOMETRIC energy
2 [F, G] = cartpole_field(x, p);
3 xdot = F + G*u;
4 worstField = max(worstField, abs(gradE.'*xdot - u*x(3)));
```

Measured: 5.7×10^{-14} W against a 460 W scale with the corrected equations, and 337 W with the old ones. The same file also checks that the linearization about $q_2 = 0$ is oscillatory with $\omega^2 = (m_1 + m_2)g/(Lm_1) = 5.880000$ (hanging is *stable*), that $q_2 = \pi$ is a real unstable pair, and that an unforced flight conserves E (1.6×10^{-12} J over 3 s).

Where it breaks / what is assumed

The general rule, worth carrying to any physics code: **at least one test must not be a mirror**. If every check compares the implementation against another expression of the same implementation, the suite measures internal consistency and nothing else. Conservation laws, analytic limits, equilibrium character and dimensional analysis are all cheap sources of an outside opinion.

10 Where the code lives

path	what it is
ex2_cart_pole_swing_up/try2/	the direct method: objective, defects, dynamics help
cartpole_common/cartpole_field.m	f and g , the one home for the physics
ex3_cart_pole_pmp/cartpole_pmp_rhs.m	the 8-state PMP field
ex3_cart_pole_pmp/cartpole_pmp_prop.m	propagator and variational STM
ex3_cart_pole_pmp/run_cartpole_pmp.m	seed, shoot, verify, report
ex3_cart_pole_pmp/run_tests.m	the runner that fails on failure
oclib/+oc/ms_bvp.m	the shared multiple-shooting engine
oclib/+oc/duals_to_costates.m	the covector rules (§5)
orbit_transfer/doc/algorithms_orbit_transfer.tex	the same ideas at campaign scale

Reading order for someone new. Run `run_cartpole_pmp` and watch it print; read `cartpole_pmp_rhs.m` (four lines, the whole principle); then `run_cartpole_pmp.m` top to bottom; then `test_cartpole_physics.m`, which is the shortest statement of what it means to actually believe a dynamics model.

11 The executable companion, and the objective that never has to choose

ELI5

Everything this document described is also a script you can run: `cartpole_minenergy_study.m`. Press Run and it works through the same story in the same order, printing PASS or FAIL as it goes instead of only at the end. One more thing worth noticing on the way out: at no point did the best push ever require a decision. It fell out of a formula, every time, at every instant. The next two chapters of this project — not written yet — change exactly that: give the cart a top speed of push it may not exceed, or make it pay for every newton instead of every newton squared, and the same question stops having a formula for an answer and starts having a switch.

Intuition

The script is not organized section-for-section with this document — it is organized as a pipeline, load → seed → solve → check — so here is the correspondence:

- §1 (what problem is this) ↔ the script’s steps 0 and 2: the tolerance table, then the same cost and boundary conditions, in code.
- §2 (the plant) ↔ step 1: `cartpole_params`, and the physics oracle (V1) run and asserted *first*, before anything downstream is allowed to mean anything.
- §4–§6 (the indirect method, the bridge, shooting) ↔ steps 3 and 4: the seed lifted verbatim from `run_cartpole_pmp.m`’s own multiplier block, then `oc.ms.bvp`.
- §7 (what the two methods produce) ↔ the X1 check folded into step 6.
- §8 (how we know it is right) ↔ steps 5–8: the independent-seed check (X2), the necessary conditions (N1–N6), the sufficiency test (S1–S2), and the cross-check against the library instrument.
- §9 (the sign error) ↔ step 1’s V1 oracle again — the same outside-the-equations check, run once and relied on everywhere after.

Why no switch here: an unconstrained control being minimized against a *strictly convex* cost is a parabola with nowhere to hit a wall. Put a wall in front of it (a bound on u) or straighten the parabola into a line or a V (a cost linear in u or in $|u|$) and the minimizer stops living where calculus finds it and starts living at the wall — which is a decision, made anew whenever the wall relocates.

Rigor

Here $H(u) = u^2 + \sigma u$ (dropping the parts independent of u), with $\sigma = \lambda^\top g$ the switching function already named in §1. §4 already showed why this problem gives a formula: H is *strictly convex* in u ($H_{uu} = 2 > 0$ everywhere) and u ranges over all of $\mathbb{R}$, so the stationary point $H_u = 0$ is both necessary and sufficient for the global minimizer — one equation, one answer.

That argument leaned on two properties together: *no bound on u* , and a cost *strictly* convex in u . Both are what a different objective can change. Bound the control, or replace u^2 by a cost that is merely convex rather than strictly so (linear in u , or in $|u|$), and H no longer has an interior minimum for calculus to find: the minimizer is pushed onto the boundary of the allowed set (a wall) or onto the one point where a piecewise-linear cost fails to be smooth (a corner) — and which wall or corner it sits at can change during the flight. That is a switch, decided by a sign, not a derivative. Minimum time (control bounded, cost t_f) and minimum fuel (control bounded, cost $\int |u| dt$) are exactly the two objectives in this family that do this. Locating and tracking the switch — including the saltation matrices a jump in structure requires — is sub-projects B and C's content, and arrives with their code; their equations are not repeated here.